

One Class, Three New Superpowers

Overview

This lab adds three new capabilities to the Person class you've been building since Lab01 (name and email, stored UPPERCASE / lowercase, with a constructor, mutators, accessors, and toString). Instead of three separate programs, this time everything lives in one evolving Person.h — you'll extend the same class in three stages, and a single main() demonstrates each stage in its own labeled block: a static data member that every Person shares (Experiment 1), a static member function that doesn't need any Person object to call it (Experiment 2), and friend functions that are granted access to Person's private data without being members of Person at all (Experiment 3).

Learning Objectives

- Explain the difference between a member that belongs to each object and one that belongs to the class as a whole.
- Declare a static data member inside a class and correctly define it exactly once outside the class.
- Write and call a static member function both through an object and through the class name.
- Explain why a static member function has no this pointer, and what that does and doesn't let it access.
- Declare a friend function and understand exactly what access it grants (and to what — not to the whole world, just to that one function).
- Write operator<< as a friend function — the standard C++ reason this feature exists in the first place.

Static vs. Ordinary Members, at a Glance

	Ordinary member	Static member
How many copies exist?	One per object	Exactly one, shared by the whole class
Needs an object to access?	Yes, always (p.getName())	No — ClassName::member also works
A static function can use it?	Never (no this pointer)	Yes

Experiment 1 — A Static Data Member

Add a private static member to `Person` that tracks how many `Person` objects currently exist: `objectCount`. Increment it in both constructors; add a destructor that decrements it.

Member	Purpose
<code>static int objectCount;</code>	Declared inside the class, like any private member — but it is not stored inside any individual <code>Person</code> object. There is exactly one, shared by the entire class.
<code>int Person::objectCount = 0;</code>	The required out-of-class definition — exactly one line, outside the class, in <code>Person.h</code> . A static member is only declared inside the class; it must be defined somewhere too, or the linker can't find it.
<code>~Person()</code>	The destructor. Decrements <code>objectCount</code> whenever a <code>Person</code> goes out of scope or is deleted, keeping the count accurate.

Note. Declaring

```
static int objectCount;
```

inside the class is not enough by itself. If you forget the initialization statement (written outside the class)

```
int Person::objectCount = 0;
```

your code will compile just fine (the compiler only needs the declaration to check your syntax) and then fail at the *link* step, with an error along these lines:

```
undefined reference to `Person::objectCount'
```

A Preview of what we want to accomplish

Create three `Person` objects — put the third one inside its own nested `{ }` block so it goes out of scope (and its destructor fires) before the others do — and ask the count through different objects to see that they all report the same shared value:

```
===== Experiment 1: Static Data Member =====
After creating p1: 1
After creating p2: 2
After creating p3 (nested scope): 3
After p3 goes out of scope, asking p1: 2
Same count asked through p2 instead : 2
```

Notice the last two lines: `p1` and `p2` were never touched, yet asking *either* one for the count reports 2 — because `objectCount` isn't really "`p1`'s" or "`p2`'s" copy, it's the class's one and only copy. That's the entire idea of a static data member in one observation.

Experiment 2 — A Static Member Function

Add a public static function to Person:

```
static int getObjectCount() { return objectCount; }.
```

A static member function is not tied to any particular object, so it can be called two ways — through an object, the way you'd call any other member function, or through the class name alone, with no object anywhere in sight:

Member	Purpose
<code>p.getObjectCount()</code>	Ordinary call syntax, through an existing object. Works, but requires you to have a Person lying around just to ask a question that has nothing to do with any specific Person.
<code>Person::getObjectCount()</code>	Static call syntax, through the class name. No object required — this can be called even while zero Person objects exist anywhere in the program.

End Experiment 1's block so `p1` and `p2` also go out of scope (bringing the count back to 0), then immediately call `Person::getObjectCount()` before creating anything new, to prove no object is needed:

```
===== Experiment 2: Static Member Function =====
No Person objects exist right now, yet we can still ask:
Person::getObjectCount() = 0
After creating p4, both syntaxes agree:
  p4.getObjectCount()      = 1
  Person::getObjectCount() = 1
```

Observation - Why can't a static function read name or email the way an ordinary member function can?

Every ordinary (non-static) member function secretly receives a hidden **this** pointer to the specific object it was called on — that's how `getName()` knows *which* Person's name to return. A static function has no object to be called "on", so it has no **this** pointer, and therefore no way to know which object's name or email you could possibly mean. Try adding this (broken) method to Person and compile it, to see the compiler catch exactly that:

```
static string badIdea() {
    return name;    // ERROR: which Person's name?
}
```

```
error: invalid use of member 'Person::name' in static member function
    return name;
           ^~~~
```

Remove `badIdea()` again once you've seen the error — it isn't part of the final `Person.h`.

Experiment 3 — Friend Functions

A **friend** function is declared inside a class but defined outside of it like an ordinary stand-alone function (not with a `Person::` prefix — it is not a member). Declaring it a friend grants that one specific function, and only that function, permission to reach into `Person`'s private data. Add the following declaration to the `Person` class.

Member	Purpose
<code>friend bool isSameEmail(const Person& a, const Person& b);</code>	An ordinary friend function (no operator overloading involved) that compares two <code>Person</code> objects' emails directly.

Inside `isSameEmail` code use `a.email` and `b.email` directly (not `a.getEmail()` and `b.getEmail()`) — that direct access to private data is exactly what being a friend buys you. Then the friend function:

```
===== Experiment 3: Friend Functions =====
Using the friend function isSameEmail():
    isSameEmail(p4, p5) = false
    isSameEmail(p4, p6) = true
```

(`p6` is a third `Person` constructed with the email "`LISA@GMAIL.COM`" — `setEmail` lowercases it to the same `lisa@gmail.com` that `p4` has, which is why that comparison comes back true.)

To see exactly what friend is granting, try writing a *second* version of `isSameEmail` as an ordinary function that `Person` did **not** declare as a friend:

```
// NOT declared 'friend' inside Person
bool isSameEmailBad(const Person& a, const Person& b) {
    return a.email == b.email;    // ERROR: email is private
}
```

```
error: 'std::string Person::email' is private within this context
    return a.email == b.email;
           ^~~~~
```

Being a free function (not a member) isn't what's blocked here — plenty of free functions exist that use `Person` perfectly well through `getName()/getEmail()`. What's blocked is direct access to the private fields specifically, and friend is the one thing that lifts that block, for exactly the function(s) named in the friend declaration and nothing else.

Deliverables

1. Person.h — the `INLINE` styled class, extended with all three experiments' features.
2. main.cpp — the test driver (do not modify) exercising all three experiments in sequence.
3. The two intentionally-broken snippets (`badIdea()` and `isSameEmailBad()`) are for you to try and observe, not part of the final Person.h — remove them before concluding this project.
4. Compile cleanly with no warnings.

Quality Control

- `objectCount` is declared static and private inside Person, and defined exactly once outside the class.
- Both constructors increment `objectCount`; the destructor decrements it.
- `getObjectCount()` is declared static and returns the correct shared count, whether called via an object or via `Person::`.
- `isSameEmail` is declared friend inside Person and defined outside it as a free function (no `Person::` prefix).
- `isSameEmail` access `p.name/p.email` (or `a.email/b.email`) directly, not through `getName()/getEmail()`.
- The program's output matches the three transcripts above (object count values, both call syntaxes agreeing, and the two `isSameEmail` results).

Stretch Goal (optional)

Add a second friend function, `mostLikelyRelated(const Person& a, const Person& b)`, that returns true when a and b's emails share the same domain (the part after `@`) even if the full address differs.

Starter Code - Person.h

```
// Person.h
// CS2 Lab04 - Static Members and Friend Functions
// INLINE Single-compilation file format (there is no Person.cpp.)
#pragma once
#include <iostream>
#include <sstream>
using namespace std;

class Person {
private:
    string name;
    string email;

    // TODO (Experiment 1): declare a private static int called
    // objectCount. It is declared here like any other member, but it is
    // NOT part of any individual Person object - there will be exactly
    // one objectCount shared by the whole class no matter how many
    // Person objects exist.

public:
    Person() {
        name = "";
        email = "";
        // TODO (Experiment 1): increment objectCount.
    }
    Person(string name, string email) {
        setName(name);
        setEmail(email);
        // TODO (Experiment 1): increment objectCount.
    }
    // TODO (Experiment 1): write a destructor (~Person()) that
    // decrements objectCount.

    void setName(string name) {
        for (size_t i = 0; i < name.length(); i++) {
            name[i] = toupper((char)(name[i]));
        }
        this->name = name;
    }
    void setEmail(string email) {
        for (size_t i = 0; i < email.length(); i++) {
            email[i] = tolower((char)(email[i]));
        }
        this->email = email;
    }
};
```

```

}
string getName() const { return name; }
string getEmail() const { return email; }

string toString() const {
    ostringstream sout;
    sout << this << " Person [Name: " << name << ", Email: " << email << "];
    return sout.str();
}

// TODO (Experiment 2): write a PUBLIC STATIC member function
// static int getObjectCount();
// that returns objectCount. Remember: inside a static function
// there is no 'this' pointer, so it can only touch other static
// members - it could not, for example, also return name.

// TODO (Experiment 3): declare FRIEND function here (this goes
// inside the class body, but it is not a member):
// friend bool isSameEmail(const Person& a, const Person& b);
};

// TODO (Experiment 1): DEFINE objectCount here, exactly once, outside
// the class, e.g.:
// int Person::objectCount = 0;
// A static data member is only DECLARED inside the class; forgetting
// this line compiles fine but fails at link time with an error like
// "undefined reference to Person::objectCount".

// TODO (Experiment 3): define the friend function here, as an
// ordinary free function (no Person:: prefix - it is not a member).
// Because Person declared it a friend, it may read p.name / p.email
// (or a.email / b.email) directly, even though those are private.
//
// bool isSameEmail(const Person& a, const Person& b) {
//     ...
// }

```

Starter Code - main.cpp

```
// main.cpp - CS2 C14 Lab04 - Static Members and Friend Functions (SOLUTION KEY)
#include <iostream>
#include "Person.h"
using namespace std;

int main() {
    cout << "==== Experiment 1: Static Data Member =====>> endl;
    {
        Person p1("Homer Simpson", "homer@aol.com");
        cout << "After creating p1: " << p1.getObjectCount() << endl;

        Person p2("Marge Simpson", "marge@aol.com");
        cout << "After creating p2: " << p2.getObjectCount() << endl;

        {
            Person p3("Bart Simpson", "bart@aol.com");
            cout << "After creating p3 (nested scope): " << p3.getObjectCount() << endl;
        } // p3's destructor runs here

        cout << "After p3 goes out of scope, asking p1: " << p1.getObjectCount() << endl;
        cout << "Same count asked through p2 instead : " << p2.getObjectCount() << endl;
    } // p1 and p2's destructors run here

    cout << endl << "==== Experiment 2: Static Member Function =====>> endl;
    cout << "No Person objects exist right now, yet we can still ask:" << endl;
    cout << "Person::getObjectCount() = " << Person::getObjectCount() << endl;

    Person p4("Lisa Simpson", "lisa@gmail.com");
    cout << "After creating p4, both syntaxes agree:" << endl;
    cout << "  p4.getObjectCount()      = " << p4.getObjectCount() << endl;
    cout << "  Person::getObjectCount() = " << Person::getObjectCount() << endl;

    cout << endl << "==== Experiment 3: Friend Functions =====>> endl;
    Person p5("Maggie Simpson", "MAGGIE@GMAIL.COM");

    cout << "Using the friend function isSameEmail():" << endl;
    Person p6("Lisa Simpson", "LISA@GMAIL.COM");
    cout << boolalpha;
    cout << "  sameEmail(p4, p5) = " << isSameEmail(p4, p5) << endl;
    cout << "  sameEmail(p4, p6) = " << isSameEmail(p4, p6) << endl;
}
```